Experiment No: 2

Student Name: Prionti Das
UID: 24BAI71005
Branch: AIT-CSE
Section/Group: 24AIT_NTPP-3[B]
Semester: 5TH
Date of Performance:
Subject Name: Competitive Coding – II
Subject Code: 24CSP-305

Experiment 2.1

Aim: To implement a C++ program to find the index of a target value in a sorted array or determine the position where it should be inserted.

Objective:

- To understand the **Binary Search** technique.
- To efficiently search for an element in a sorted array.
- To find the correct insertion position of the target.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/search-insert-position/submissions/2143273237/

Given a sorted array of distinct integers nums and a target value, return the index if the target is found. If it is not found, return the index where it would be inserted to maintain the sorted order.

Algorithm:

- Initialize left = 0 and right = n - 1.
- Calculate the middle index using:
  mid = left + (right - left) / 2.
- Compare nums[mid] with target:
  - If equal, return mid.
  - If nums[mid] < target, search the right half.
  - Otherwise, search the left half.
- Continue until left > right.
- Return left as the correct insertion position.

Code:

```
#include <vector>
using namespace std;

class Solution {
public:
   int searchInsert(vector<int>& nums, int target) {
      int low = 0;
      int high = nums.size() - 1;

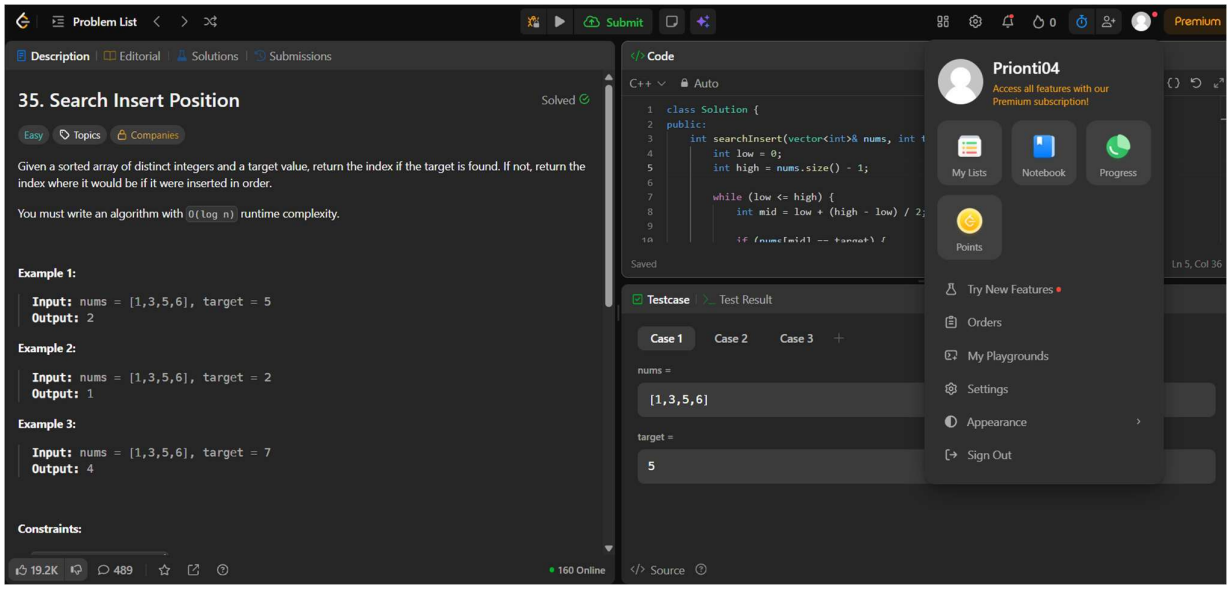
      while (low <= high) {
         int mid = low + (high - low) / 2;

         if (nums[mid] == target) {
            return mid;
```

```
            } else if (nums[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return low;
    }
};
```

Observations/Outcome: The program successfully finds the target's index or its correct insertion position using binary search.



Time Complexity: O(log n).

Space Complexity: O(1).

Learning outcomes:

- Learned how **Binary Search** works.
- Learned to find an insertion position efficiently.
- Understood how to reduce searching time from **O(n)** to **O(log n)**.
- Learned the use of left, right, and mid pointers in sorted arrays.

Experiment 2.2

Aim: To implement a C++ program to search for a target element in a rotated sorted array using binary search.

Objective:

- To understand binary search in a rotated sorted array.
- To identify which half of the array is sorted.
- To efficiently locate the target element.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/search-in-rotated-sorted-array/submissions/2143285278

Given a rotated sorted array nums and a target value, return the index of the target if it exists in the array. Otherwise, return -1.

Algorithm:

- Initialize `left = 0` and `right = n - 1`.
- Calculate the middle index `mid`.
- If `nums[mid] == target`, return `mid`.
- Check which half is sorted:
  - If `nums[left] <= nums[mid]`, the **left half is sorted**.
  - Otherwise, the **right half is sorted**.
- Determine whether the target lies within the sorted half.
- Eliminate the half where the target cannot exist.
- Repeat until the target is found or `left > right`.
- Return `-1` if the target is not present.

Code:

```
#include <vector>
using namespace std;

class Solution {
public:
    int search(vector<int>& nums, int target) {
        int low = 0;
        int high = nums.size() - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;


            if (nums[mid] == target) {
                return mid;
            }


            if (nums[low] <= nums[mid]) {

                if (target >= nums[low] && target < nums[mid]) {
                    high = mid - 1;
                } else {
```

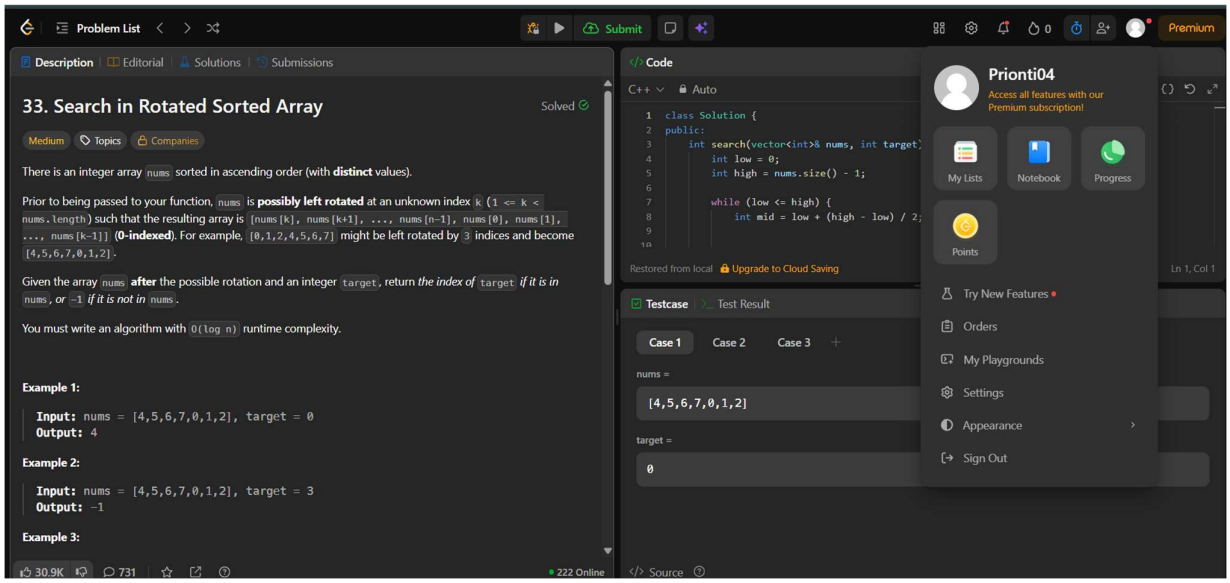
```
                low = mid + 1;
            }
        }

        else {

            if (target > nums[mid] && target <= nums[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }


    return -1;
  }
};
```

Observations/Outcome: The program successfully searches for a target element in a rotated sorted array using an efficient binary search approach.



Time Complexity: O(log n)

Space Complexity: O(1)

Learning outcomes:

1. Learned how to apply **Binary Search** to rotated sorted arrays.
2. Learned to identify the sorted portion of an array.
3. Understood how to eliminate unnecessary search ranges.
4. Learned to achieve **O(log n)** search time with constant extra space.